Universidad de Antioquia
Facultad de Ingeniería

Informática II – Semestre 2026-2

Desafío II

Amigos para siempre, con Ude@Book

Informe preliminar y Análisis de Diseño

Juan Martín López González

Juan Camilo David Zapata

09 de Octubre de 2026

1. Contextualización

Este documento presenta el análisis y diseño previo para desarrollar Ude@Book, un simulador de red social. El mayor reto del proyecto consiste en administrar de forma eficiente la información de un número n de usuarios, utilizando únicamente la memoria RAM. Para lograrlo, aplicaremos los principios de la programación orientada a objetos (POO) en C++ y manejaremos la memoria dinámica de forma totalmente manual, ya que el uso de la librería estándar (STL) no está permitido.

2. Análisis del problema y decisiones de diseño

2.1. Gestión de archivos y memoria secundaria

Como tenemos estrictamente prohibido usar el disco duro para guardar datos temporales, el programa deberá cargar toda la información desde los archivos hacia nuestras estructuras dinámicas justo al arrancar el sistema.

En cuanto a guardar los progresos, actualizar los archivos de texto por cada mínima acción volvería el sistema sumamente lento debido al costo de lectura y escritura. Teniendo en cuenta que para este desafío la seguridad y persistencia estricta de los datos no es un factor crítico, decidimos priorizar totalmente la eficiencia. Por esto, el sistema operará completamente en RAM y la actualización de los archivos hacia el disco se hará únicamente cuando el usuario lo solicite de manera manual mediante una opción en el menú, o al momento de cerrar la sesión podría ser una opción.

2.2. Estructuras de datos: matriz de adyacencia

Para manejar las conexiones entre los usuarios, decidimos usar una matriz de adyacencia a través de un arreglo dinámico de dos dimensiones. Elegimos esta opción porque nos permite saber de manera casi instantánea cuántos amigos en común tienen dos personas. Esto acelera muchísimo la función de sugerencias de amistad, evitando sobrecargar el procesador sin importar cuántos usuarios haya registrados en el sistema. El resto de los elementos, como los mensajes o las fotos, los administraremos creando clases que controlen sus propios arreglos dinámicos.

2.3. Prevención de fugas de memoria

Trabajar con punteros requiere bastante cuidado. Por eso, establecimos como regla liberar la memoria inmediatamente después de que un bloque de datos ya no se necesite. Como medida extra de seguridad, cada vez que liberemos un puntero lo igualaremos a nullptr. Así evitamos dejar referencias sueltas que puedan causar que el programa colapse de la nada mientras el usuario interactúa con el menú.

3. Diseño General de la Solución

3.1. Arquitectura de Módulos

Dividiremos el proyecto de la siguiente forma:

- **main.cpp:** Será el punto de inicio del programa, donde se creará la red social y se ejecutará el menú principal.
- **UdeaBook.h / .cpp:** Funcionará como el núcleo del sistema, administrando el arreglo principal de usuarios y la matriz de amistades.
- **Usuario.h / .cpp:** Agrupará la información personal de cada cuenta, sus solicitudes pendientes y las clases encargadas de sus fotos y mensajes.
- **Dependientes:** Tendremos archivos separados para Mensaje, Album y Foto. A cada uno le programaremos su propio constructor de copia para evitar cruces o errores de memoria.

3.2. Estrategia para el reporte de consumo de recursos

Para mostrar qué tan eficiente es cada función, haremos lo siguiente:

- **Consumo de memoria:** Sumaremos el tamaño base de las clases usando sizeof() junto con un cálculo matemático de la memoria dinámica que hemos ido reservando. Esto nos permitirá imprimir en pantalla la cantidad exacta de bytes que el programa está utilizando en ese instante.
- **Conteo de iteraciones:** Vamos a crear variables como contadores dentro de los métodos más importantes. Estas variables irán sumando uno por cada ciclo for o while que ocurra, y si llamamos a una función externa, le sumaremos sus respectivas iteraciones a nuestro conteo total.

4. Diagrama de clases (Resumen)

Para cumplir con el formato de UML simplificado que vimos en clase, el diagrama que entregamos en esta etapa muestra únicamente los nombres de las entidades y cómo se relacionan entre sí (con su direccionalidad y cardinalidad).

La arquitectura parte de una clase principal llamada RedSocial, que contiene a todos los usuarios mediante una relación de pertenencia con cardinalidad de 1 a 0..∗. A su vez, la clase Usuario tiene una relación con ella misma denominada Amigos>cardinalidad de 0..∗ a 0..∗. Por dentro, cada Usuario es dueño de sus propias listas de Mensaje, SolicitudAmistad y Album todas con cardinalidad de 1 a 0..∗. Por último, cada Album se compone de entidades de tipo Foto cardinalidad de 1 a 0..∗.